COM3306 · OPERATING SYSTEMS · 3 CREDITS · EXAM: 3 HOURS, ANSWER ALL 5

Operating Systems – Gap Notes & Worked Answers

Your five lessons cover introduction, processes, deadlock, IPC and memory. These notes fill what the past papers ask but the lessons don't cover: **file management**, the **Banker's algorithm**, **I/O topics** (BIOS, DMA, interrupts), plus fully worked answers for the calculations that come every year.

---

1. Files and access methods
2. Directory structures
3. File allocation and i-nodes
4. Free memory: bitmaps vs linked lists
5. File permissions and protection
6. Banker's algorithm (worked)
7. CPU scheduling Gantt charts (worked)
8. LRU page replacement (worked)
9. BIOS, DMA, I/O addressing, interrupts
10. Past paper map

1. Files and access methods

A **file** is a named collection of related information stored on secondary storage. The OS's **file system** decides how files are named, stored, organised, protected and accessed.

- **Attributes:** name, identifier, type, location (disk blocks), size, protection (owner, permissions), timestamps (created, modified, accessed).
- **Operations:** create, open, read, write, seek (reposition), close, delete, truncate.

Access methods

Method	How it works	Example
Sequential	Records are read one after another in order, like a tape.	Reading a log file, compilers, media players
Direct (random / relative)	Any block can be read or written directly by its block number, in any order, without reading the ones before it.	Databases, airline reservation records
Indexed	An index (like a book index) holds pointers to blocks; first search the index, then jump to the block.	Large databases, library catalogues

සිංහලෙන්: Sequential = පොතක් මුල ඉඳන් අගට කියවනවා වගේ, පිළිවෙළට. Direct = පොතේ 250 පිටුවට කෙලින්ම පනිනවා වගේ – block number එක දන්නවා නම් කෙලින්ම යන්න පුළුවන්, ඉක්මනුයි (database වලට හොඳයි). Indexed = පොතේ index එක බලලා පිටුව හොයාගෙන යනවා වගේ.

2. Directory structures

Structure	Idea	Good	Bad
Single-level	One directory for all files of all users.	Very simple, fast to implement.	Every file name must be unique for everyone → **naming conflicts**; no grouping; no privacy; searching gets slow as files grow – does not scale.
Two-level	A master directory with one user directory (UFD) per user.	Different users can use the same file name; users are isolated (security); faster search inside a user's own directory.	Users cannot group their own files into folders; sharing between users is awkward.
Tree (hierarchical)	Directories inside directories, one root; absolute and relative paths.	Natural grouping, efficient search by path, each user has a home subtree, permissions per directory.	A file cannot appear in two places (no direct sharing); deleting a directory must handle its subtree.
Acyclic graph (DAG)	A tree that also allows **shared** subdirectories/files through links, but no cycles.	Sharing without copying: one copy, many paths → no inconsistent copies, saves space.	Same file has several names (aliasing); deletion needs reference counts to avoid dangling links.
General graph	Links allowed anywhere, cycles possible.	Most flexible.	Cycles cause infinite loops in searches; needs garbage collection.

2026 Q5(d) – multi-user system: direct access, tree and DAG (7 marks)

- **Preventing data conflicts:** a **tree** gives every user a separate home directory, so two users can both have `report.txt` without clashing. A **DAG** lets users share one real copy of a file through links, so there are no out-of-date duplicate copies.
- **Improving efficiency: direct access** lets a program jump straight to the record it needs instead of reading the whole file – fast for databases with many users. A tree shortens searches (only one directory path is searched); a DAG avoids storing duplicates.
- **Maintaining security:** permissions can be set on each directory and file in the tree (owner/group/others), so users cannot reach each other's private folders. In a DAG, only the users given a link (and permission) can reach a shared file; reference counts stop a file being deleted while others still use it.

2025 Q5(d) – single-level vs two-level: efficiency and scalability

A single-level directory must be searched in full for every lookup and all names must be unique, so it becomes slow and unmanageable as users and files grow – **poor scalability**. A two-level directory searches only one user's directory (**faster**), allows the same name for different users and isolates users, so it **scales better** – but still offers no sub-folders for organising many files.

> **සිංහලෙන්:** Single-level = හැමෝගෙම files එකම folder එකක – නම් ගැටෙනවා, files වැඩි වෙද්දී හොයන්න අමාරුයි. Two-level = හැම user කෙනෙක්ටම තමන්ගේම folder එකක්. Tree = folders ඇතුළේ folders (අපේ computer එකේ වගේ). DAG = tree එකක්ම, ඒත් එක file එකක් links හරහා තැන් කිහිපයක පෙන්නන්න පුළුවන් – copy කරන්නේ නැතුව share කරනවා.

---

3. File allocation and i-nodes

Method	How	Advantage	Disadvantage
Contiguous	File stored in consecutive blocks; directory keeps start block + length.	Fast sequential and direct access.	External fragmentation; hard to grow a file.
Linked	Each block holds a pointer to the next block (FAT keeps the pointers in a table).	No external fragmentation; files grow easily.	Slow direct access (must follow the chain); a broken pointer loses the rest.
Indexed	An index block holds all the block addresses of the file.	Fast direct access, no external fragmentation.	Index block overhead for small files.

I-nodes (2023 Q5(c), 4 marks)

An **i-node (index node)** is a small fixed-size data structure used in UNIX/Linux file systems. There is one i-node per file. It stores the file's **attributes** (owner, permissions, size, timestamps, link count) and the **disk addresses** of the file's blocks: about 12 direct block pointers, plus a single-indirect, a double-indirect and a triple-indirect pointer for large files. The directory entry only maps a **file name → i-node number**.

Uses / advantages: only the i-nodes of **open** files need to be in memory (saves RAM); small files are reached quickly through direct pointers while very large files are still supported through indirect blocks; several names (hard links) can point to the same i-node, which makes sharing easy.

සිංහලෙන්: i-node එක කියන්නේ file එකක "ID card" එක වගේ – owner කවුද, permissions මොනවද, size එක, වෙලාවන්, සහ disk එකේ data blocks තියෙන තැන් (addresses) ඒකේ තියෙනවා. Directory එකේ තියෙන්නේ "file නම → i-node අංකය" විතරයි. Open කරපු files වල i-nodes විතරක් RAM එකට ගන්න නිසා memory ඉතුරුයි; ලොකු files වලට indirect pointers තියෙනවා.

---

4. Free memory management: bitmaps vs linked lists (2023 Q5(a))

Bitmap: memory is divided into equal allocation units; each unit has one bit (0 = free, 1 = used). The bitmap has a fixed size that depends only on memory size and unit size. Simple, but to place a process of k units the OS must search the bitmap for k consecutive 0 bits, which is slow. Small units → big bitmap; big units → wasted space in the last unit.

Linked list: a list of segments, each either a process (P) or a hole (H), with start address, length and a pointer to the next entry, kept sorted by address. Allocation uses first fit, next fit, best fit or worst fit; when a process ends, neighbouring holes are merged easily. The list grows and shrinks with the number of segments, and searching can be faster than scanning bits.

	Bitmap	Linked list
Space used	Fixed, known in advance	Varies with number of holes/processes
Finding free space	Slow – scan for a run of 0s	Faster – walk the hole entries
Freeing memory	Just clear bits	Merge with neighbouring holes
Simplicity	Very simple	More complex (pointers)

සිංහලෙන්: Bitmap = memory එකේ හැම කොටසකටම bit එකක් (0 හිස්, 1 පිරිලා). Size එක ස්ථිරයි, ඒත් ලොකු process එකකට හිස් තැනක් හොයන්න 0 දිගටම තියෙන තැනක් scan කරන්න ඕනේ – හෙමින්. Linked list = "process කොටසක් / හිස් කොටසක්" කියලා ලැයිස්තුවක්; හිස් තැන් හොයන්න ඉක්මන්, process එකක් ඉවර වුණාම අල්ලපු හිස් තැන් එකතු කරන්න ලේසියි.

5. File permissions and protection (2025 Q5(c), 4 marks)

File system permissions are rules attached to each file and directory that say **who** may do **what** with it. In UNIX/Linux there are three rights – **read (r)**, **write (w)**, **execute (x)** – for three classes of users – **owner (u)**, **group (g)**, **others (o)**.

How they control access: every time a process tries to open, change or run a file, the OS compares the process's user ID and group ID with the file's owner, group and permission bits, and allows or refuses the operation. Access Control Lists (ACLs) extend this with per-user rules (used by Windows NTFS).

Example: `-rwxr-x--- report.sh` → owner can read, write, execute; group can read and execute; others have no access. In numbers: `chmod 750 report.sh` (r = 4, w = 2, x = 1 → 7 = rwx, 5 = r-x, 0 = ---).

> **සිංහලෙන්:** Permissions කියන්නේ file එකකට "කාටද මොනවද කරන්න පුළුවන්" කියන නීති. r = කියවන්න, w = වෙනස් කරන්න, x = run කරන්න. Owner, group, others කියලා කොටස් 3කට වෙන වෙනම දෙනවා. File එකක් open කරන හැම වෙලාවකම OS එක මේ bits බලලා ඉඩ දෙනවද නැද්ද කියලා තීරණය කරනවා.

6. Banker's algorithm

The **Banker's algorithm** is a **deadlock-avoidance** algorithm. Before granting a request, the OS checks whether the system would still be in a **safe state** – a state where there is at least one order (safe sequence) in which every process can get its maximum need and finish. If the request leads to an unsafe state, the process waits.

Safety algorithm steps

1. **Need = Max - Allocation** for every process.
2. **Work = Available**. Mark every process "not finished".
3. Find a not-finished process whose **Need ≤ Work** (every resource type).
4. Pretend it finishes: **Work = Work + its Allocation**; mark it finished; add it to the safe sequence.
5. Repeat steps 3–4. If all processes finish → **SAFE** (give the sequence). If you get stuck → **UNSAFE**.

2026 Q4(d) – worked answer (8 marks)

Available = **(3, 3, 2)**. Step 1 – Need table:

Process	Allocation			Max			Need = Max - Alloc		
	A	B	C	A	B	C	A	B	C
P0	0	1	0	7	5	3	**7**	**4**	**3**
P1	2	0	0	3	2	2	**1**	**2**	**2**
P2	3	0	2	9	0	2	**6**	**0**	**0**
P3	2	1	1	2	2	2	**0**	**1**	**1**
P4	0	0	2	4	3	3	**4**	**3**	**1**

Step 2 – Work = Available = (3, 3, 2), then the safety check (scan P0…P4 in order each round):

Step	Process	Need ≤ Work?	New Work = Work + Allocation
1	P1	(1, 2, 2) ≤ (3, 3, 2) ✓	(3, 3, 2) + (2, 0, 0) = **(5, 3, 2)**
2	P3	(0, 1, 1) ≤ (5, 3, 2) ✓	(5, 3, 2) + (2, 1, 1) = **(7, 4, 3)**
3	P4	(4, 3, 1) ≤ (7, 4, 3) ✓	(7, 4, 3) + (0, 0, 2) = **(7, 4, 5)**
4	P0	(7, 4, 3) ≤ (7, 4, 5) ✓	(7, 4, 5) + (0, 1, 0) = **(7, 5, 5)**
5	P2	(6, 0, 0) ≤ (7, 5, 5) ✓	(7, 5, 5) + (3, 0, 2) = **(10, 5, 7)**

All processes can finish → the system is in a **SAFE** state. Safe sequence: **⟨P1, P3, P4, P0, P2⟩**.

2025 Q4(c) – worked answer

Available = **(1, 1)**. Step 1 – Need table:

Process	Allocation		Max		Need = Max - Alloc	
	A	B	A	B	A	B
P1	1	2	3	3	**2**	**1**
P2	2	1	4	2	**2**	**1**
P3	1	1	2	2	**1**	**1**

Step 2 – Work = Available = (1, 1), then the safety check (scan P0…P2 in order each round):

Step	Process	Need ≤ Work?	New Work = Work + Allocation
1	P3	(1, 1) ≤ (1, 1) ✓	(1, 1) + (1, 1) = **(2, 2)**
2	P1	(2, 1) ≤ (2, 2) ✓	(2, 2) + (1, 2) = **(3, 4)**
3	P2	(2, 1) ≤ (3, 4) ✓	(3, 4) + (2, 1) = **(5, 5)**

All processes can finish → the system is in a **SAFE** state. Safe sequence: **⟨P3, P1, P2⟩**.

Since a safe sequence exists, the system is **not deadlocked**.

RAG method vs Banker's algorithm (2025 Q4(b))

	Resource Allocation Graph	Banker's algorithm
Works for	Single instance of each resource type	Multiple instances of each resource type
How	Draw processes, resources and claim/request/assignment edges; a cycle means deadlock (single instance)	Matrices (Allocation, Max, Need, Available) and the safety algorithm
Cost	Simple, visual, cheap	More computation; processes must declare their maximum need in advance
Common	Both are used for deadlock avoidance/detection and both keep the system out of unsafe states	

Major disadvantage of Banker's (2023 Q4(b)): every process must state its **maximum** resource need in advance, the number of processes and resources must be fixed, and running the safety check on every request is expensive – so it is rarely practical in real systems.

> **සිංහලෙන්:** මුලින්ම Need = Max - Allocation table එක හදන්න. ඊට පස්සේ Work = Available. "මගේ Need එක Work එකට වඩා අඩු හෝ සමානද?" කියලා process එකින් එක බලන්න. හරි නම් ඒ process එක ඉවර වෙනවා කියලා හිතලා ඒකේ Allocation එක Work එකට එකතු කරන්න. ඔක්කොම process ඉවර කරන්න පුළුවන් නම් SAFE, ඒ පිළිවෙළ තමයි safe sequence එක. හැම පියවරකම Work එක ලියලා පෙන්නන්න – marks තියෙන්නේ එතන.

---

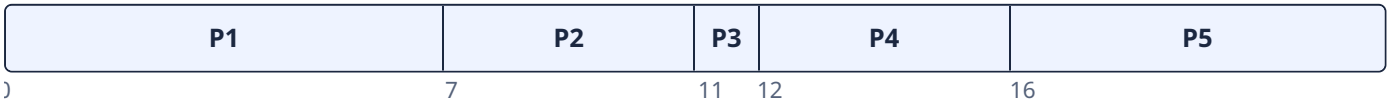
7. CPU scheduling – Gantt charts and averages

```
Completion time (CT) = time the process finishes
Turnaround time (TAT) = CT - Arrival time
Waiting time (WT) = TAT - Burst time
```

Round Robin rule used here (state it in your answer): when a new process arrives at the same moment a running process's quantum ends, the **new arrival joins the ready queue first**, then the preempted process.

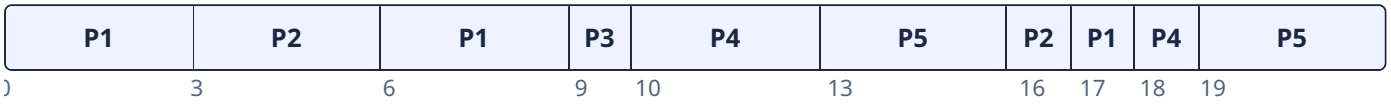
2026 Q2(d): P1(0,7), P2(2,4), P3(4,1), P4(5,4), P5(6,6)

FCFS



Process	Arrival	Burst	CT	TAT = CT - AT	WT = TAT - BT
P1	0	7	7	7	0
P2	2	4	11	9	5
P3	4	1	12	8	7
P4	5	4	16	11	7
P5	6	6	22	16	10
Average			**13.60**	**10.20**	**5.80**

ROUND ROBIN, QUANTUM = 3 MS



Process	Arrival	Burst	CT	TAT = CT - AT	WT = TAT - BT
P1	0	7	18	18	11
P2	2	4	17	15	11
P3	4	1	10	6	5
P4	5	4	19	14	10
P5	6	6	22	16	10
Average			**17.20**	**13.80**	**9.40**

Which suits an interactive system? Round Robin – every process gets the CPU within a short, bounded time (quantum), so users see quick responses, even though the average waiting time here is higher. FCFS lets a long job block everyone (convoy effect).

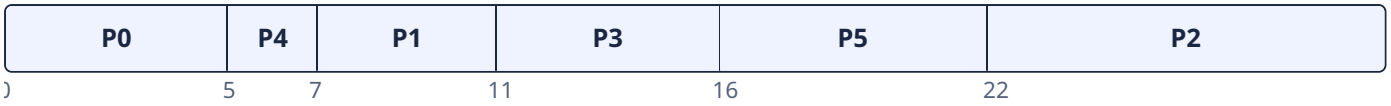
2025 Q2(d): P0(0,5), P1(1,4), P2(1,9), P3(3,5), P4(4,2), P5(5,6)

FCFS



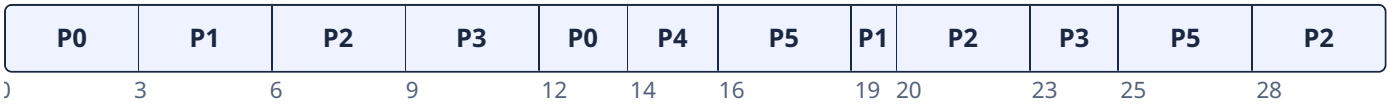
Process	Arrival	Burst	CT	TAT = CT - AT	WT = TAT - BT
P0	0	5	5	5	0
P1	1	4	9	8	4
P2	1	9	18	17	8
P3	3	5	23	20	15
P4	4	2	25	21	19
P5	5	6	31	26	20
Average			**18.50**	**16.17**	**11.00**

SJF (NON-PREEMPTIVE)



Process	Arrival	Burst	CT	TAT = CT - AT	WT = TAT - BT
P0	0	5	5	5	0
P1	1	4	11	10	6
P2	1	9	31	30	21
P3	3	5	16	13	8
P4	4	2	7	3	1
P5	5	6	22	17	11
Average			**15.33**	**13.00**	**7.83**

ROUND ROBIN, QUANTUM = 3 MS



Process	Arrival	Burst	CT	TAT = CT - AT	WT = TAT - BT
P0	0	5	14	14	9
P1	1	4	20	19	15
P2	1	9	31	30	21
P3	3	5	25	22	17
P4	4	2	16	12	10
P5	5	6	28	23	17
Average			**22.33**	**20.00**	**14.83**

සිංහලෙන්: FCFS = ආපු පිළිවෙළට. SJF (non-preemptive) = CPU එක හිස් වුණාම, ඒ වෙලාවට ඇවිත් ඉන්න අයගෙන් burst time එක අඩුම එකා. Round Robin = හැමෝටම quantum එකක් (3 ms) ගානේ වාරෙට; ඉවර නැත්නම් queue එකේ අගට යනවා. Gantt chart එක ඇඳලා, ඊට පස්සේ CT, TAT = CT - arrival, WT = TAT - burst table එක හදලා average ගන්න.

---

8. LRU page replacement (2023 Q5(d), 8 marks)

Reference string: 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with **4 page frames**. LRU replaces the page that has **not been used for the longest time**.

Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	3
Frame 1	**7**	7	7	7	7	**3**	3	3	3	3	3	3	3	3
Frame 2		**0**	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3			**1**	1	1	1	1	**4**	4	4	4	4	4	4
Frame 4				**2**	2	2	2	2	2	2	2	2	2	2
Fault?	F	F	F	F	–	F	–	F	–	–	–	–	–	–

Page faults = **6** (the first 4 fill empty frames, then 3 replaces 7 and 4 replaces 1). Hits = 8.

Check yourself: with 3 frames the same string gives 9 faults.

සිංහලෙන්: Page එකක් frames වල නැත්නම් page fault (F). Frames පිරිලා නම්, වැඩිම කාලයක් පාවිච්චි නොකරපු (least recently used) page එක අයින් කරනවා – ඒ සඳහා ආපස්සට බලලා අන්තිමට පාවිච්චි කළේ කවදද කියලා බලන්න. මෙතන faults 6යි.

9. BIOS, DMA, I/O addressing and interrupts

Why do computers need a BIOS? (2023 Q2(a))

The **BIOS** (Basic Input/Output System) is firmware stored in ROM/flash on the motherboard. When the power is switched on, RAM is empty, so the CPU needs a fixed program to start from. The BIOS: (1) runs the **POST** (power-on self-test) to check RAM, keyboard and disks; (2) **initialises the hardware**; (3) finds the boot device and **loads the bootloader**, which loads the operating system; (4) provides basic low-level I/O routines and the setup (CMOS) settings. Modern PCs use **UEFI**, which does the same job.

DMA (2023 Q4(d))

Direct Memory Access: a DMA controller moves a block of data between an I/O device and memory **without the CPU copying each byte**. The CPU only sets up the transfer (memory address, byte count, direction) and receives one interrupt when it is finished.

Increases concurrency: while the DMA controller transfers data, the CPU runs other processes – CPU work and I/O happen at the same time.

Complicates hardware: needs an extra DMA controller; the CPU and DMA compete for the memory bus (bus arbitration, cycle stealing); cache contents can become inconsistent with memory; more interrupt handling.

Separate I/O space vs memory-mapped I/O (2023 Q3(c))

With a **separate I/O address space** (port-mapped, special IN/OUT instructions): the **whole memory address space stays free for RAM**; I/O instructions are easy to recognise and can be restricted to the kernel (protection); device registers are never cached by mistake; address decoding for memory is simpler. (Memory-mapped I/O instead uses normal load/store instructions on device registers, which is more flexible but uses up memory addresses and needs caching disabled for those addresses.)

Precise vs imprecise interrupts (2025 Q5(e), 5 marks)

A **precise interrupt** leaves the machine in a well-defined state: (1) the program counter is saved in a known place; (2) all instructions before it have **fully completed**; (3) **no instruction after it** has started (or its effects are undone); (4) the state of the instruction at the PC is known. The OS can simply restart from the saved PC.

An **imprecise interrupt** does not meet these conditions – in pipelined or superscalar CPUs, several instructions may be half-finished. The CPU dumps a lot of internal state for the OS, so interrupt handling becomes **slow and complex**.

Why busy waiting is a poor way to get mutual exclusion (2023 Q3(b))

A process that busy-waits keeps **using CPU time while doing nothing useful** (spinning in a loop). It can also cause the **priority inversion problem**: a high-priority process spins forever while a low-priority process inside the critical section never gets the CPU to leave it. Blocking (sleep/wakeup, semaphores) is better.

> **සිංහලෙන්:** BIOS = computer එක on කළාම මුලින්ම run වෙන program එක – hardware check කරලා (POST), OS එක load කරනවා. DMA = CPU එකට කරදර නොකර device එකයි RAM එකයි අතර data කෙලින්ම යවන controller එක; ඒ අතරේ CPU එකට වෙන වැඩ කරන්න පුළුවන්. Precise interrupt = interrupt එක ආව වෙලාවේ machine එකේ තත්ත්වය පැහැදිලියි (කලින් instructions ඉවරයි, පස්සේ ඒවා පටන් ගත්තේ නෑ).

---

10. Past paper map

Topic	2023 Jan	2025 Feb	2026 Mar	Where to study
OS basics, kernel, architectures	Q1	Q1	Q1	Lesson 1
Processes, schedulers, Gantt charts	Q2(c), Q3(a)	Q2	Q2	Lesson 2 + §7 here
IPC, critical section, semaphores, Peterson	Q2(b), Q3(b)	Q3	Q3	Lesson 4
Deadlock, Banker's	Q4	Q4	Q4	Lesson 3 + §6 here
Memory, paging, page replacement	Q3(d), Q5	Q4(e), Q5(a)(b)	Q5(a)–(c)	Lesson 5 + §4, §8 here
Files, directories, permissions	Q5(c)	Q5(c)(d)	Q5(d)	§1–§5 here
I/O: BIOS, DMA, interrupts	Q2(a), Q3(c), Q4(d)	Q5(e)	–	§9 here

---

Study notes written for your Semester 3.2 exam plan from the past papers in your folder. Check anything that looks different from what your lecturer taught.